



TECHNISCHE
UNIVERSITÄT
WIEN

ottobock.

Framework for System Simulation

Thesis – Introduction

Florian Frech, 18.07.2025

1. Introduction and Motivation

2. Modeling and Simulation of Systems

1. Systems, Modeling Approaches and Classification
2. Principles of Modeling and Building Models
3. Mathematical Representation
4. Differential Equations and Numerical Integration

3. Equation-based Modeling

1. Physical Modeling Paradigm
2. Causality, Structural Singularity and Partition Algorithm
3. Example: Structural Singular DAE System
4. Index and Initialization of DAE Systems & Pantelide's Algorithm
5. Numerical Solutions of DAEs
6. Hybrid Systems
7. Equation-based Modeling with Modelica

4. Integration of heterogeneous Models

1. Overview Functional Mock-Up Interface (FMI)
2. FMI Standard 3.0
3. Co-Simulation: It's about time
4. Methods and Tools for Co-Simulation

4. Analysis of Simulink

1. Core Modeling Concepts and Simulation Management
2. User Interfaces & Visualizations
3. Relevant Concepts

5. System Simulation Framework

1. Managing Software Complexity
2. Architectural Framework Approach
3. V-Model for System Development
4. Requirements, Tests and Continuous Integration

6. Demo Model Ideas

7. Further Procedure

Context – System Simulation of mechatronic Systems

- Complex mechatronic systems like prostheses combine mechanical, electrical, and control subsystems
- Subsystems are often modeled using different tools, each specialized in its domain (large domain-specific libraries, specialized functions and solvers)
 - **Modelica**: multi-domain physical modeling
 - **Simulink**: control systems
 - **FEM**: structural simulation
 - **OpenSim**: musculoskeletal dynamics
- Complex interaction and coupling of subsystems
- System behavior depends on the interactions between these heterogeneous models

Objective

- Develop a Python-based simulation framework that
 - supports object-oriented modeling
 - allows equation-based modeling
 - enables co-simulation / integration of heterogeneous subsystems
- Provide templates for physical and logical templates
- Facilitate reusability, visualization, and model exchange between tools

Motivation

- System simulation is crucial for the design, validation, and optimization of complex mechatronic systems
- Domain specific tool do not directly enable interoperability / combination of subsystems
- Flexible framework is required to integrate heterogeneous subsystem models, support modular design and enable scalable simulation

Challenges

- Handling computational causality, algebraic loops, DAE index
- Supporting event-based hybrid systems (continuous + discrete dynamics)
- Managing different time-resolutions and model granularities

Systems, Modelling Approaches, and Classification

Definition of a System

- System as an object or collection of objects whose properties we want to study
- Group of interacting elements with defined boundaries and rules
- Interacts with the environment via inputs (u) and outputs (y)
- Modeled as a set of interacting variables influencing environment
- Observer dependent choice of inputs / outputs

Black-, White-, Grey-Box Models

- Black box: no physical insight needed
- White-box: based on fundamental physical laws and principles, fully described
- Grey-box combines both, physical structure + parameters from data

Model Purpose

- System analysis and synthesis
- Enables simulation, control, and optimization
- Allows replacing physical experiments when infeasible, expensive, dangerous, etc.

System Decomposition

- Subsystem simplify modeling and enable modular reuse
- Component-based modeling and hierarchical structure are crucial for managing system complexity

Classification A vs. B	Type A	Type B
Static vs. Dynamic	Instantaneous relationship (steady) state, algebraic equations	Time-dependent dependent behavior, ODEs or DAEs
Continuous vs. Discrete Time	Evolves continuous over time using differential equations	Time progresses in discrete states, quantities change at certain time instants
Lumped vs. Distributed	Quantity is approximated by being lumped together (point mass)	Physical distribution of the quantity (heat transfer rod)
Deterministic vs. stochastic	Predictable output for given input	Includes randomness (noise, uncertainty)
Quantitative vs. Qualitative	Variable values can be represented numerically	No numerical representation naturally, maybe rough classification into finite set, discrete time

Principles of Modeling

- 1. Principal of Correctness:**
A model needs to represent the facts correctly regarding structure and dynamics (semantics). Specific notation rules must be considered (syntax).
- 2. Principle of Relevance:**
All relevant items must be modeled. Non-relevant items must be left out, i.e., the value of the model does not decline if these items are removed.
- 3. Principle of Cost vs. Benefit:**
The amount of effort to gather the data and produce the model must be balanced against the expected benefit.
- 4. Principal of Clarity:**
The model must be understandable and usable. The required knowledge for understanding the model should be as low as possible.
- 5. Principle of Comparability:**
A common approach to modeling ensures future comparability of different models that have been created independently from each other.
- 6. Principle of Systematic Structure:**
Models produced in different views should be capable of integration. Interfaces need to be designed to ensure interoperability.

Building Models

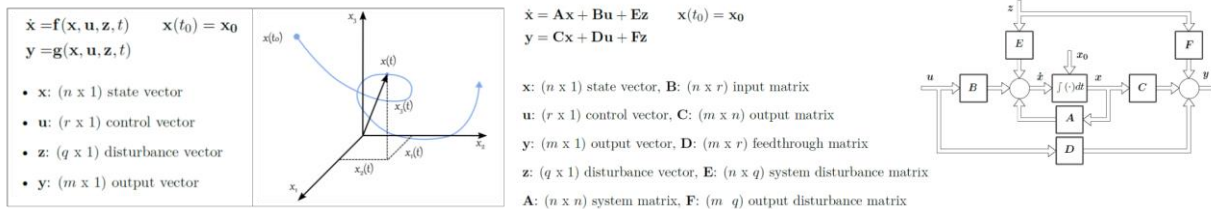
- System-related knowledge required for building mathematical models
 - Application expertise: application area and facts related to specific model application
- 1. Identify the main components** of a system and the kinds of interactions between them
 - 2. Divide component into subcomponents:** each parts fits the description on an existing model from some model library, or we can use appropriate laws of nature / relationship to describe behavior
 - 3. State the component interfaces**
 - 4. Make a mathematical formulation of the interactions** between the components of the model

(Pannek, 2023); (Fritzson, 2015.)

Mathematical Model Representation

State Space Models

- describe dynamical system which can be characterized by a **finite number of state variables**
- State x is an element of the **state space**
- States of a system can be represented as a point within the n -dimensional state space
- Trajectory: curves of all these **points in the state space for varying time** in an interval



General Constrained Systems / DAEs

- Mix of **dynamic (differential)** and **constraint (algebraic)** equations
- Arise in **constrained** mechanics, electrical circuits, and multi-body stems
- Essential that the algebraic variables do not appear in the system in the form of the derivatives
- System is **split into differential equations and algebraic equations**
- Algebraic equations are formed by conservation laws and constraints

Distributed-Parameter Model

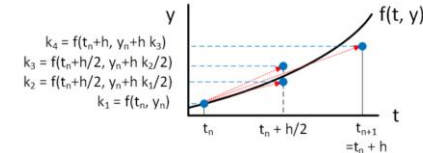
- Governed by PDEs
- Example: Beam deflection, heat conduction, etc.

Differential Equations and Numerical Integration

- Differential equations arise naturally when modeling physical systems
- Depending on the system, simulation may require static, continuous, discrete or triggered time modeling

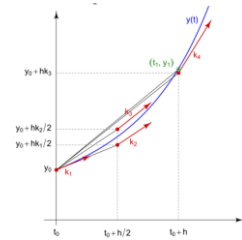
Explicit Methods

- Compute next steps using only the current state (one-step)
- Pros: Easily implemented, intermediate steps used
- Cons: Requires multiple function evaluations, limited stability



Implicit Methods

- Solving equations involving future state (via solving a system of equations)
- Pros: better stability, can handle stiff systems
- Cons: More complex, require solving algebraic equations per step



Adaptive Step Size

- Dynamically adjusts step-size based on estimated error (ode45)
- Pros: efficient for varying dynamics, reduces the evaluations
- Cons: Requires two embedded methods, consistency depends on lower order method

Deployment Challenge & Grid Adaptation

- Unified time grid:** All subsystems use a common step size (simple but may be computationally expensive)
- Split Grids:** Allows individual step size but requires synchronization logic
- Debouncing:** Holds values constant between updates to avoid unnecessary evaluations
- Multiscale Grids:** Enables co-simulation of fast and slow subsystems (fast sensors vs. slow mechanical motion)

Physical Modeling Paradigm (Acausal Modeling)

- **Acausal** models describe physical interactions **independent of signal direction** (no predefined input/output).
- Based on physical laws (e.g., Kirchhoff, Newton, conservation laws).
- **Variables are connected through equations**, not signal flow.
- Enables reusability and flexibility in system configuration.
- Modelica is a key example of an equation-based language.
- Supports object-oriented component modeling and multi-domain simulation.

Computational Causality

- Global property of the model, that describes which form the equation is **used to evaluate the unknown variable**
- The way in which equations are written does not inform about their computational causality
- Causality **depends on the equation itself and on other equations and algorithms of the model**
- Assumption: all variables that appear differentiates can be selected as state variables and the model has an equal number of DoFs and state variables
- Structural analysis identifies solvable sets of equations using the incidence matrix (structural Jacobian) which indicates **which unknown variables intervene in each equation**

Structural Singularity

- Is the number of unknown variables the same as the number of equations?
- Checked by finding a sequence of permutations of the matrix that allows to obtain a permuted matrix with all the element on the main diagonal are nonzero

Reasons for Structural Singularity

- Number of equations E is equal to the number of unknown variables V, but it is **not possible to obtain a matrix with zero-free main diagonal** by permuting the columns of the incidence matrix
- Model may be mathematically incorrect
- Model is mathematically correct but the number of its degrees of freedom is smaller that the number of variables that appear differentiated in the model
 - $E > V$: **overdetermined**
 - $E < V$: **underdetermined**

Partition Algorithm

- Transforming the **incidence matrix into a block lower triangular (BLT)** matrix with diagonal block of minimal size
- Unknown variables that intervene in the equation of each block are calculated from the equations in the block or from equations in previous blocks
- **1 x 1 block**: model variables can **be evaluated in sequence** one after another
- **n x n block**: set of n variables is **calculated solving a system of n simultaneous equations**
- **Tarjan's algorithm**: employ **bipartite graphs to represent the computational structure** of the model

Overdetermined and Underdetermined Systems

- Overdetermined: set of dummy variables are introduced which intervene in all equations
 - BLT: all dummy variables are in the first diagonal block
- Underdetermined: set of dummy equations are introduced which contains all the unknown variables (all dummy equations will be in the last diagonal block)

1. Computational structure	2. Original Incidence Matrix (structural Jacobian)	3. Permuted Incidence Matrix	4. Partition 1: swapping rows	5. Partition 2: solved computational causality	6. Evaluation order
$\begin{aligned} f_1(x, y, z) &= 0 \\ f_2(x, \dot{x}, y) &= 0 \\ f_3(x, z) &= 0 \end{aligned}$	$\begin{matrix} & \text{der } x & y & z \\ \begin{matrix} f_1 \\ f_2 \\ f_3 \end{matrix} & \begin{pmatrix} 0 & X & X \\ X & X & 0 \\ 0 & 0 & X \end{pmatrix} \end{matrix}$	$\begin{matrix} & y & \text{der } x & z \\ \begin{matrix} f_1 \\ f_2 \\ f_3 \end{matrix} & \begin{pmatrix} X & 0 & X \\ X & X & 0 \\ 0 & 0 & X \end{pmatrix} \end{matrix}$	$\begin{matrix} & z & y & \text{der } x \\ \begin{matrix} f_3 \\ f_1 \\ f_2 \end{matrix} & \begin{pmatrix} \boxed{X} & 0 & 0 \\ X & X & 0 \\ 0 & X & \boxed{X} \end{pmatrix} \end{matrix}$	$\begin{matrix} & z & y & \text{der } x \\ \begin{matrix} f_3 \\ f_1 \\ f_2 \end{matrix} & \begin{pmatrix} \boxed{X} & 0 & 0 \\ X & \boxed{X} & 0 \\ 0 & X & \boxed{X} \end{pmatrix} \end{matrix}$	$\begin{aligned} f_3(x, [z]) &= 0 \\ f_1(x, [y], z) &= 0 \\ f_2(x, [\text{der } x], y) &= 0 \end{aligned}$

(Urquía & Martín, 2018)

3. Example: Structurally Singular DAE System

Description:

- Two liquid storage tanks are connected in parallel to a source of liquid
- ρ : Density
- F_V : Volumetric Flow Rate
- S_i : Cross-sectional area of the tanks
- $C_i = \frac{S_i}{\rho \cdot g}$ (known constant)

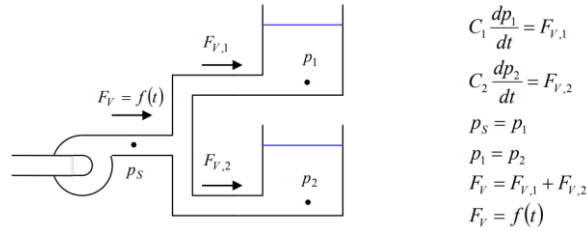


Figure 5.1: Two tanks connected in parallel to a liquid source.

- Replace derivatives by dummy variables: $\frac{dp_1}{dt} \rightarrow derp_1$ $\frac{dp_2}{dt} \rightarrow derp_2$
- Assumption: Two **variables that appear differentiated (p_1 and p_2) can be selected as state variables**. This allows the classification in known and unknown variables as follows:
 - Known: $t, C_1, C_2, p_1, p_2, f(t)$
 - Unknown: $F_{V,1}, F_{V,2}, F_V, p_S, derp_1, derp_2$

- The **resulting incidence matrix is singular**, as all the elements of the 4th row are zero.

	$F_{V,1}$	$F_{V,2}$	F_V	p_S	$derp_1$	$derp_2$
$C_1 \cdot derp_1 = F_{V,1}$	X	0	0	0	X	0
$C_2 \cdot derp_2 = F_{V,2}$	0	X	0	0	0	X
$p_S = p_1$	0	0	0	X	0	0
$p_1 = p_2$	0	0	0	0	0	0
$F_V = F_{V,1} + F_{V,2}$	X	X	X	0	0	0
$F_V = f(t)$	0	0	X	0	0	0

- Analyzing computational causality by applying the rules of the partition algorithm:

- $p_S = p_1$ contains only the unknown p_1
- $F_V = f(t)$ contains only the unknown F_V
- $derp_1$ and $derp_2$ only appear in one equation each, moved to last rows
- $p_1 = p_2$ does not contain any unknown
- One equation with 3 unknown

	p_S	F_V	$F_{V,1}$	$F_{V,2}$	$derp_1$	$derp_2$
$p_S = p_1$	X	0	0	0	0	0
$F_V = f(t)$	0	X	0	0	0	0
$p_1 = p_2$	0	0	0	0	0	0
$F_V = F_{V,1} + F_{V,2}$	0	X	X	X	0	0
$C_1 \cdot derp_1 = F_{V,1}$	0	0	X	0	X	0
$C_2 \cdot derp_2 = F_{V,2}$	0	0	0	X	0	X

- Partition algorithm fails because the model only has one DoF, but we have assumed two. Connecting the tanks imposes their bottom pressure two be equal ($p_1 = p_2$). This reduces the DOFs. **Initial value of one pressure determines the other pressure.**

6. Solution approach

Assumption: $p_1(t) = p_2(t)$ is equivalent to impose the equality of the initial values $p_{1,0} = p_{2,0}$ and the derivatives $der p_1 = der p_2$.

Approach: Adding to the model the derivative of the constraint that reduces the number of DoF and select one of the pressures as state variable (e.g., p_1)

6. Resulting computational causality:

- $F_{V,1}, F_{V,2}, derp_1, derp_2$ are calculated by solving the system of **simultaneous equations**.

p_1	state variable
$[F_V]$	$f(t)$
$[p_S]$	p_1
$p_1 = [p_2]$	← constraint
$C_1 \cdot derp_1 = F_{V,1}$	
$C_2 \cdot derp_2 = F_{V,2}$	
$derp_1 = derp_2$	← derivative of the constraint
$F_V = F_{V,1} + F_{V,2}$	

Procedure for Simulating Structurally Singular DAE Systems

- Time derivatives are replaced by dummy variables (unknown, must be calculated from equations)
- Certain model equations are differentiated symbolically a certain number of times, and these differentiated equations are added to the model. No equation is removed.
- The number of variables selected as state variables is equal to the number of DoF of the model.
- Variables selected as state variables are calculated by numerical integration of their derivatives. State variables are therefore classified as known for assigning computational causality.
- Variables that appear differentiated in the model and have not been selected as state variables are classified as unknown and have to be calculated from the model equations.

1. Which model equations have to be differentiated how many times?

2. Is there a criteria to calculate the number of DoF of a DAE system?

- Equations are differentiated if the new equations obtained provide additional useful information.
- Pantelides algorithm allows to determine which model equations have to be differentiated and how many times.

- Modelica implement **variants of the Pantelides algorithm, symbolic formula manipulation and simplification algorithms**, and **algorithms for selecting the state variables**.

- Complete process is performed automatically

(Urquía & Martín, 2018)

Index and Initialization of DAE Systems

- Classification of the model variables into known and unknown is made assuming that all the variables that appear differentiated can be selected as state variables
- Incorrect for models with several degrees of freedom smaller than the number of variables that appear differentiated
- Number of degrees of freedom (DoF) of a model is the number of time-dependent variables whose initial value can be set independently
- State variables are classified as known for the assignment of computational causality and variables that appear differentiated but are not state variables have to be calculated from model equations

Index of DAE Systems

- Minimum number of times that all part of the system equation must be differentiated w.r.t. time t to determine dx/dt as a continuous function of x, t
- Measures the mathematical complexity: how many times one need to differentiate the algebraic constraints to reduce the system to a set of ordinary differential equation (to expose all hidden derivatives and remove implicit constraints)
- Numerical algorithms don't work well for DAE index ≥ 2 . Require symbolical manipulation to reduce the index order before solution is computed.

Initialization of DAE Systems

- Initial conditions must satisfy the original DAE system at the initial time t_0 (not sufficient cond.)
- Additional (hidden) constraints on the initial values are obtained by differentiating certain equations of the original DAE
- Existence of hidden constraints is hidden to the structural singularity of the DAE system

Procedure for DAE with Hidden Constraints

1. Hidden constraints are added to the system equations (= extended DAE system)
 - same solution and DoFs as the original system but index of the extended DAE system is lower and thus less difficult to solve numerically
2. Number of DoF of the extended DAE is calculated
 - will be smaller than the number of variables that appear differentiated in the original system
3. Computational causality of the extended DAE system is assigned
 - selecting as many state variables as DoFs the system has

The Pantelides Algorithm

- Analyzes the structure of the DAE system to find the minimum subset of equations whose differentiation introduces additional constraints on the initial value vector
- Algorithm employs bipartite graphs for representing the computational structure of the DAE system
 - Does not require performing arithmetic operations or symbolic differentiation
 - Algorithm converges if the DAE system is well posed
- Direct Problem: evolution of the input variable is known
 - Objective: calculate the evolution of the output variables
- Inverse Problem: Desired evolution of the output variable is known
 - Objective: calculate the inputs that have to be applied for obtaining these outputs
 - Successive derivatives of equations impose additional constraints, Pantelides algorithm does not converge
 - Algorithm differentiates these equations indefinitely

Numerical Solution of DAEs

Algebraic Loops

- Set of equations that form a main diagonal block in the BLT, linear or nonlinear
- Typically arise of making simplification that a fast phenomenon occurring instantaneously

Stiff Systems

- Systems with fast and slow dynamics
- slowest time constant of the system typically determines the time scale of the system's global response
- fastest time constant affects the maximum time step length of the numerical integration method
- Explicit methods are not efficient for simulating stiff models
- Implicit integration methods allow significantly larger time steps than explicit methods

Solvers:

- DASSL: Backward Differentiation formula, variable order/step
- Inline Integration: improves performance by inlining integration into compiled model code
- Mixed-Model Integration: Implicit for stiff/fast subsystems, explicit for slower parts to balance stability vs. efficiency

(Urquía & Martín, 2018)

- State of hybrid models is described using **continuous and discrete time variables**
- Continuous change in the value of the continuous-time variables
- Instantaneous changes at events in the discrete-time variables

Omola Hybrid Model (OHM) Formalism: $M = (q, x, y, E, G, H, \Phi, \Delta)$

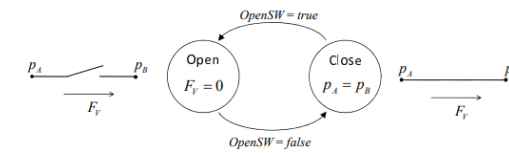
- $q = (q_1, \dots, q_n)$: vector that contains the model **discrete-time variables**
- $x = (x_1, \dots, x_n)$: vector that contains the model **continuous-time variables**
- $y = (y_1, \dots, y_n)$: vector that contains the **continuous-time algebraic variables**
- E : set with all possible types of **events**
- G : set of expressions, definition of the **continuous-time behavior** of the model
- H : set of invariant expressions describing **triggering conditions of events**
- $\Phi: H \rightarrow E$: function that associates an **event type to each invariant expression**
- $\Delta = (\delta_1, \dots, \delta_n)$: set of vector expressions that **describe the instantaneous change** in the model variables produced by the execution of an event

Simulation Algorithm:

- 1. Solution of the continuous-time problem:**
Described by a DAE system and implies solving a non-linear system of simultaneous equations and performing numerical integration. Discrete-time variables q only have constant known values and only change when event is executed.
- 2. Detection of events:**
Checking the invariant expressions during the simulation of the continuous problem. Solution of continuous problem is stopped when invariant expression changes from true to false. Iterative algorithm for finding trigger time is started.
- 3. Determination of the event trigger time:**
Iterative method finds the trigger time within the last integration step (right limit of the found interval).
- 4. Execution of the event:**
New variables calculated at the event time must be consistent with the initial values for the continuous-time problem (restart problem). The numerical solution of the continuous-time problem is resumed after executing the event.

Variable Structure Models

- Mathematical **structure may change** during simulation runs (example: ideal switch)
- Model of this type can run in different modes (each described by a particular system of equations)
- Transition between modes produces changes in the model's mathematical description
- Computational causality of the switch's constitutive relationship depends on the value of the control variable
- Assigning **computational causality** of the complete model, the switch's constitutive relationship will **be part of an algebraic loop**



Model Initialization

- Calculation of consistent model variables at the initial time
- Nonlinear-algebraic loops: iterative methods are applied using as initial guess the corresponding values provided by the developer (Initial guess: value of the variables calculated at the previous step)
- Problem: If an event produces instantaneous changes in the model state because abrupt change in the model state can make the actual solution of the algebraic loop to be too far from the initial guess

Event Detection and Handling

- **Simultaneous Events:** **Events can trigger other events** at the same time. Challenge to detect multiple simultaneous events and ensuring a conflict-free execution order. Single-Assignment-Rule to avoid simultaneous conflicting updates of the same variable.
- **Crossing Functions:** Used to **detect state events** by monitoring zero-crossings of continuous expressions.
- **Determining Event Instants:** Time events exogenous (specified by the model) and endogenous (triggered by simulation state). State events are triggered when system conditions are met.
- **Chattering:** Caused by **too many events in a short simulation interval**. Slows down the simulation due to repeated integration restarts.

Modeling Methodology & Tools

- Object-oriented physical modeling (modular and hierarchical design, equation-based, reusability)
- Multi-domain modeling: seamless integration of physical domains (electrical, mechanical, hydraulic, thermal)
- Physical Modeling paradigm: Hybrid DAEs are formed (differential, algebraic, event-driven-logic)
 - Causality is assigned automatically at compile-time (acausal)
- Event & Discrete Dynamics:
 - events are triggered by conditions (logical, temporal, state)
 - Simulation handles event detection, location, and restart problems during hybrid simulation

Object-oriented modeling concepts

- Classes and Instances: all models are classes; instantiation is implicit. Equations, not methods define the behavior
- Specialized classes: model, block, connector, partial, function, etc.; each with specific semantic
- Component-based modeling: well defined interfaces (connectors) and strong typing
- Connectors and variable types (across and through)
- Protected vs. public variables: internal logic, model interface, encapsulation and clean interface design
- Class specialization: partial, replaceable, extends, and encapsulated enable abstraction, extension, and isolation (class reuse through composition and encapsulation)

Component and Connection Mechanism:

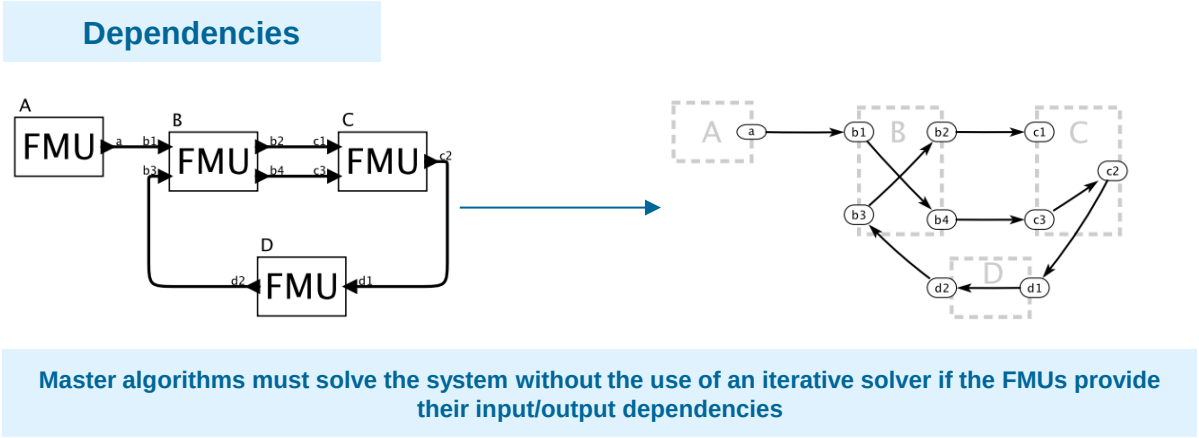
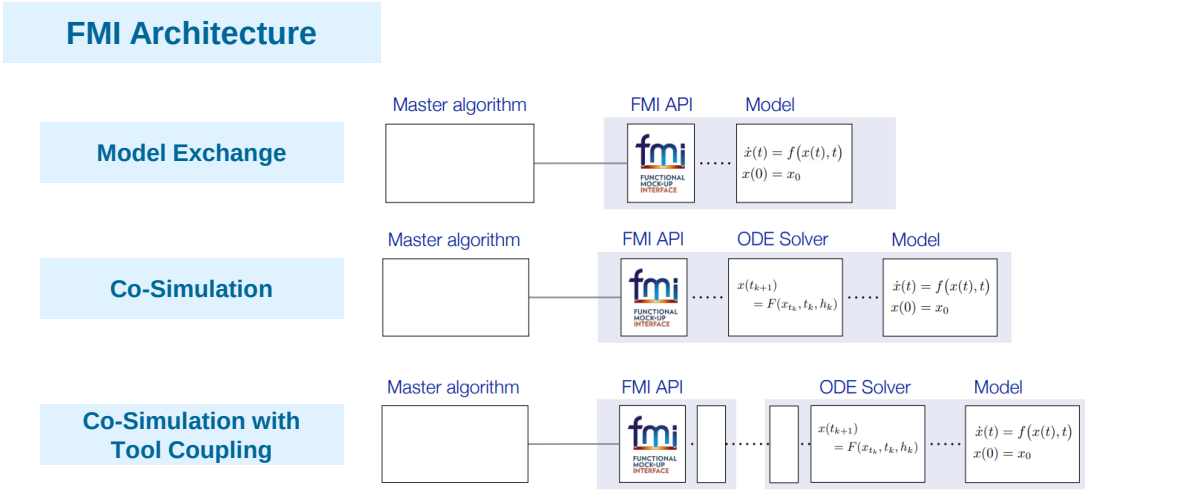
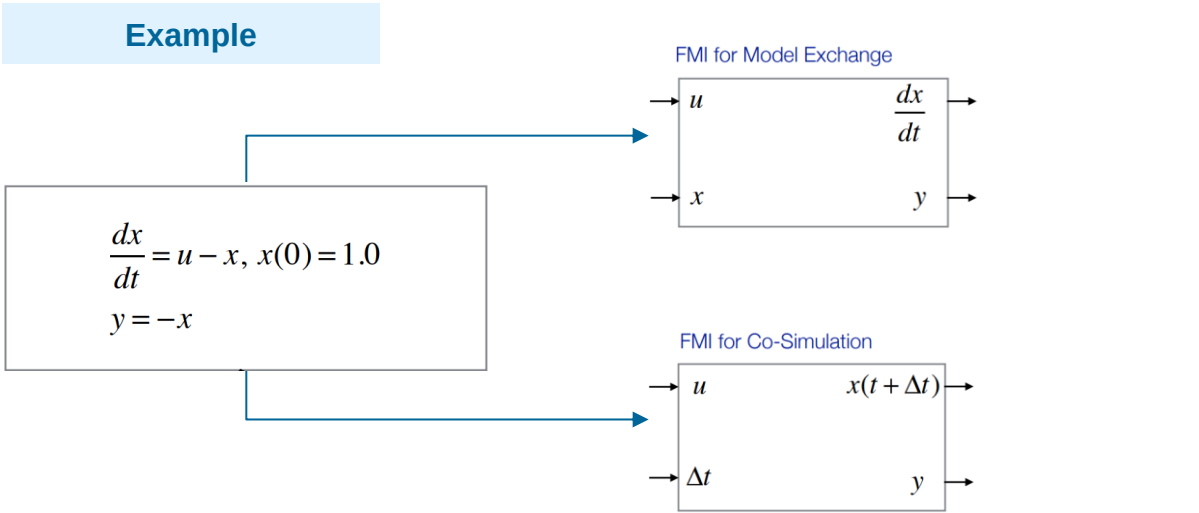
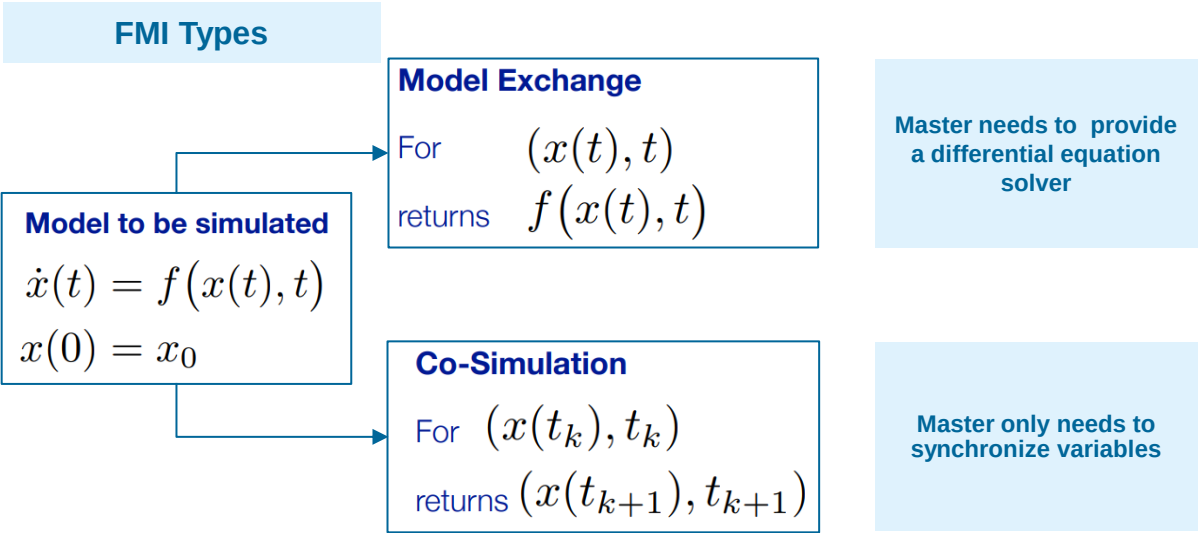
- Connectors: Carry flow and potential variables (current and voltage) and define physical interface
- Connections: Generate equations (Kirchhoff-style) automatically
- Expandable Connectors: act like information buses; allow scalable flexible component interaction
- Stream connectors: For bi-directional flow with properties (e.g., enthalpy, composition)

Execution Pipeline for Solving the DAEs

- Flattening: resolving hierarchy, inheritance
- Connection between subsystems are translated into equations
- Equation analysis & optimization
 - causality assignment
 - index reduction with Pantelide's algorithm (for DAE systems)
 - Symbolic manipulation and algebraic tearing (solving large algebraic loops)
- C code generation and linking to numerical solvers
 - Solves continuous parts with DAE solvers
 - Detect & handles events (with rollback and re-evaluation)
 - hybrid solvers for continuous and discrete logic

Relevant Concept	Explanation
Equation-based acausal modeling	- crucial for enabling modular subsystem exchange without imposing a fixed data flow - aligns with need to support heterogeneous simulation paradigms
Component Interface & Connection Concept	- Orientation on Modelica's connector concept (flow/potential) for defining clear I/O interfaces - Connection rules (sum-to-zero, equality) model physical conservation laws (coupling mechanisms)
Partial & Generic Class Mechanisms	template-based subsystem definitions in the frameworks (Motor, sensor, controller templates)
Hybrid & Multi-Domain Modelling	- critical for modeling systems involving mixed continuous/discrete dynamics - Sensor logic and mechanical actuation
Simulation Architecture & Compilation Pipeline	- Modelica's approach to flattening, index reduction, and code generation illustrates how to handle system equations systematically - Modular code generation, symbolic processing, solver integration strategies for framework
Reusable Component Libraries	Structured component libraries can guide how to organize reusable, parameterized, subsystem models
Diagrammatic Representation & Connectors	- Graphical modeling with connection diagrams - support for hierarchy visualization - Visualization of system structure and simulation flow

(Modelica Association, 2022)



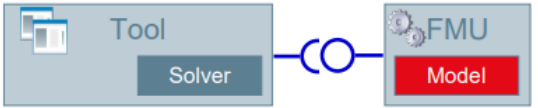
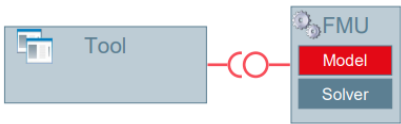
FMI Standard

- FMI-Standard defines an open interface, to be implemented by a FMU (Functional Mock-up Unit)
- FMI functions are called by a simulator that imports an FMU
- FMU may be self-integrating (Co-Simulation) or require the master tool (importer) to perform the integration (Model Exchange)
- FMI separates the description of the interface (xml) from its functionality (C API)
- FMI standardizes
 - set of C-functions, to be implemented by a model/simulator
 - an XML-model description file to be provided by a model/simulator
 - the distribution file format to be used by a model/simulator

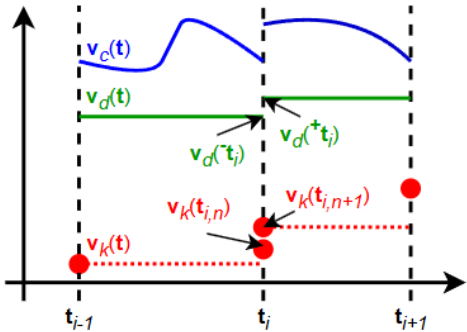
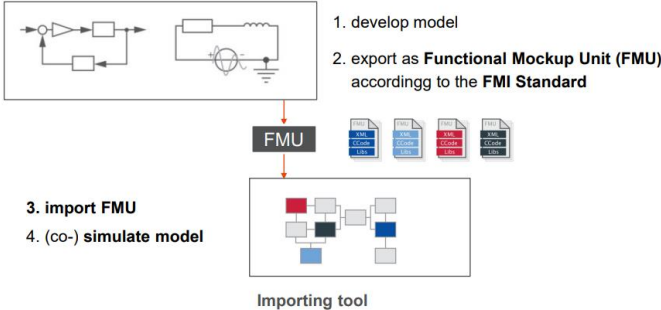
Functional Mock-up Unit

- Model container that can be distributed (zip-file with “.fmu” suffix)
- modelDescription.xml: model description with information on the model variables, interface, capabilities and model structure (limited extend)
- Model representation Binaries for one or multiple platforms and/or “black-box”, source code (C)

Interface Types

Model Exchange (ME)	Co-Simulation CS
• Importing tool provides the solver • Models are described by DAEs and discrete equations with time-, step-, and state-events • Integration algorithm (ODE or DAE solver) of the importer is responsible for advancing time, setting states, and handling events • Complex interface between importing tool and model • Importing tool must provide suitable solver for the model	• Exporting tool provides the solver • Models are exported by a modeling environment together with their solvers as runnable code • Simpler interface between importing tool and model • Freedom in the selection of co-simulation algorithm • Used in most industrial application
	

Exporting tools



Concepts

- Mathematical description of ODEs in state-space representation with discontinuities (events), algebraic system equations and discrete-time equations
- FMU and importer uses variables to exchange information (access via API functions)
- Super-dense time: $t = (t_R, t_I), t_R \in R, t_I \in N$
- State machine are used to define the legal calling sequences of FMI functions throughout the lifecycle of an FMU (correct usage of the API, consistent evaluation of model equations, safe transitions between phases like initialization, simulation, and termination) and support modular simulation
- Clocks: variables to report events or trigger events
 - Used to synchronize and control model partition execution
 - Support for periodic, aperiodic, countdown, triggered clocks
 - Input and output clocks with assigned model partitions
- Events and Event Mode
 - Time Events (scheduled), State Events (detected via indicators), and Step Events
 - Support for hybrid systems (ODEs with events)
 - Event mode allows consistent evaluation of all discontinuities and reinitializations

Motivation and Problem Statement

- Cyber-physical systems combine **discrete and continuous dynamics**
- Existing simulation tools struggle with discrete behaviors
- FMI 2.0 enables tool-coupling via co-simulation, but **lacks with robust time modeling for hybrid systems**

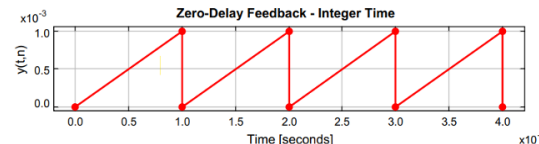
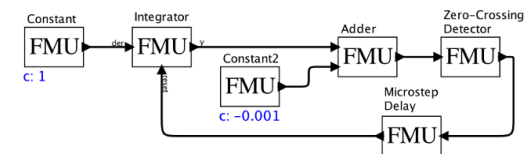
Core Issue: Inadequate Time Representation

- FMI 2.0 uses **floating-point numbers for time modeling**
- Approach cannot guarantee simultaneity of events
- Floating-point representation for time **causes inconsistencies in simulations involving discrete-time events** and instantaneous transitions

Paper Contribution: Hybrid Co-Simulation with improved Time Semantics

- Introduces a new model of time for FMI based co-simulation based on **super-dense time** and **integer representation**
- Supports continuous-time and discrete-event behavior
- Enables instantaneous reactions and zero-time events
- Ensures Causality and simultaneity across cyber and physical components

Motivating Example



- Simplest possible non-trivial illustration of the challenges of hybrid co-simulation
- Integrator (e.g., ODE solver) integrating a constant and the result being offset by a constant via an Adder (e.g., continuous-time simulation engine)
- Zero-crossing detector (e.g., discrete event processor) detects a zero-crossing of the offset output of an integrator and produces a discrete event that is then processed by the simplest cyber-component, a micro step delay, (e.g., software model of control strategy) which resets the integrator
- Simplest closed-loop cyber-physical control system
- Resetting has to occur as a discrete zero-duration discontinuity

Introduced Key Concepts

1. Supper-dense time

- Events can carry time stamp (simulation time: int t, micro step index: int n)
- Enables representing multiple discrete events at the same simulation time
- Eliminates floating point ambiguities in detecting simultaneity or causality

2. Integer Time Representation

- Using 64-bit unsigned integer with configurable resolution
- Guarantees exact arithmetic and deterministic ordering of events
- Time resolution is model specific and negotiable between FMUs and the master

3. Resolution Negotiation

- FMUs declare their preferred time resolution
- Master algorithm computes a common compatible resolution via minimum resolution / greatest common divisor
- Allows mixing FMUs operating on different time scales

4. FMU Categorization

- 0A/0B: legacy FMUs with/without zero-step support
- 1-4: varying support for resolution negotiation
- Wrappers are used to abstract FMU-specific time models and interface with the master

5. Wrapper-based Architecture

- Master delegates time and signal translation to wrapper layers
- All wrappers expose a uniform interface using integer time
- Ensures modularity and compatibility across legacy and new FMUs

6. Time conversion & Quantization

- Carefully handles conversion between:
- FP time to / from integer time
- Different integer time resolutions
- Ceiling/flooring rounding strategies to ensure causality and avoid Zeno effects
- Quantization still exists but is bounded and controlled

Background:

- Model Exchange (= strong coupling): Importer has access to model equations, possible to evaluate the equations directly, equations can be assembled and solved by a standard integration algorithm
- Co-Simulation (= weak coupling): exchange of information of inputs/outputs only at discrete points, model equations are hidden, models contain an integrator

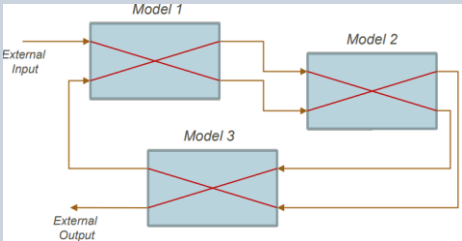
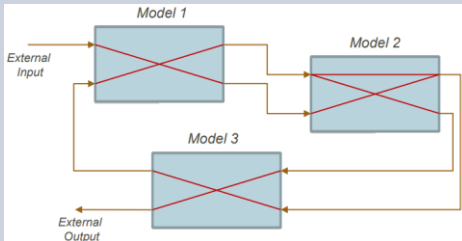
Benefits of weak coupling (co-simulation)

- Allows **tailored integrators** for component models
- Allows component **models to be run in parallel**
- Models may **have different time scales** which can be exploited by the internal integrators

Challenges of weak coupling (co-simulation)

- Numerical stability
- Synchronization
- Consistent communication

Initialization of Coupled System:

Direct feed-through	Algebraic Loop
 Possible to find a path from the external input to the external output without the need for solving the algebraic equation.	 Not possible to find a direct path from the external input to the external output without the need for solving the algebraic equation. due to the additional dependency on the outputs in Model 2.

Structural Analysis using Graph Theory

- Goal: solve the initialization problem by finding an evaluation order (direct sequence by forward evaluation or a subset of equations needs to be solved together in an algebraic loop)
- Initialization problem can be studied by considering a **directed graph $G(V,E)$**
 - Set of nodes V and set of edges E where the edges are ordered pairs of nodes
 - Each component of the output vector y and input vector u is a node
 - Edge from input $u_k^{[i]}$ to output $y_j^{[i]}$ exists if the derivative of the output equation of component j of subsystem i w.r.t. $u_k^{[i]}$ being non-zero (models the data flow/signal influence within the system)
 - Edge from output y_j to u_i exists if there is a feedback or control influence (when an output is used to compute or modify an input, common in close-loop simulations)
- **Strongly Connected Components (SCC):**
 - Subset of **mutually interdependent variables (a loop)**
 - Input and output of the component can influence other variables/parts of the system.
 - Indicates algebraic loops which require global solving or implicit methods.
 - Each SCC is solved as unit or simplified by breaking dependencies with dummy variables

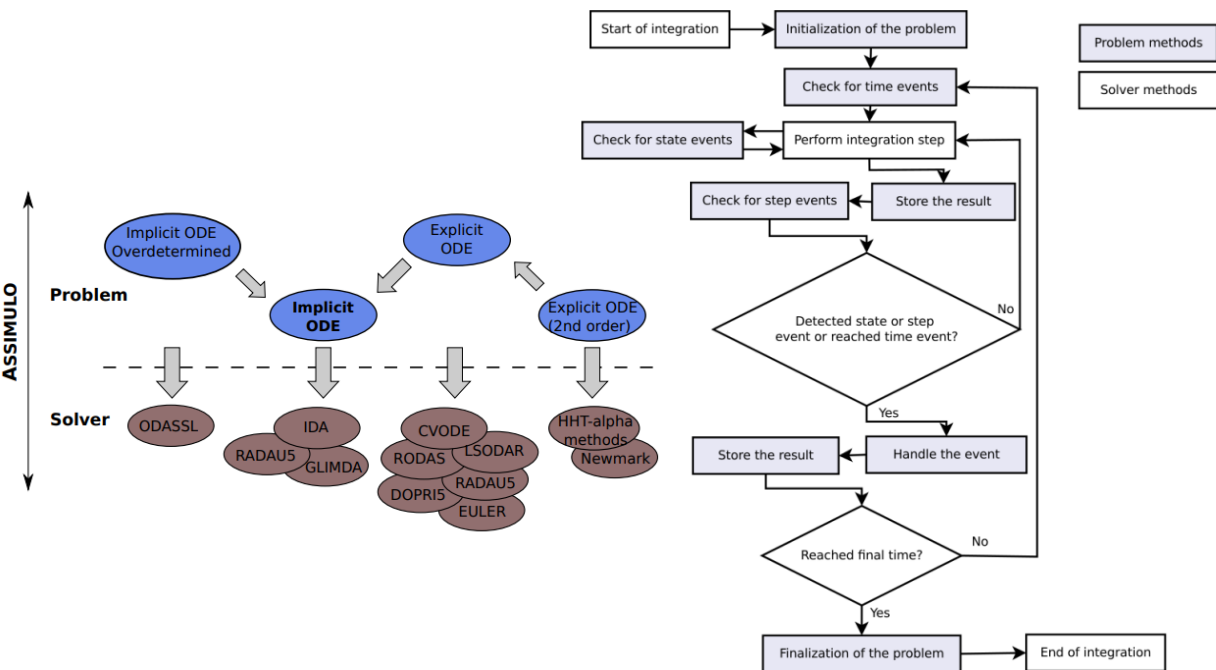
$$\begin{array}{c}
 y = g(x, u) \\
 u = c(y)
 \end{array}
 \rightarrow
 \begin{array}{c}
 y = \hat{g}(x) + Du \\
 u = Ly
 \end{array}
 \rightarrow
 y = \hat{g}(x) + DLy$$

- $u = c(y)$: coupling function, u : input, y : output, x : state variable
- g is linearly in u , and c is linear in y
- If there are **no SCC an explicit sequence can be found** to solve the above equation
- Requires that the **dependency matrix DL is nilpotent** ($DL^k = 0$ for some k)
 - there are then **no circular dependencies**
 - the spectral radius $\rho(DL) = 0$ which is important for stability of coupled systems
- System is structurally well posed if there are no algebraic loops and an explicit evaluation sequence exists
- Otherwise, **simultaneous solving of coupled systems is required** (iterative solver, index reduction)

(Andersson, 2016)

Assimulo: Solver Framework for ODEs and DAEs

- Python based simulation interface for integrating differential equations (ODEs and DAEs)
- Focuses on **unifying various solvers under a consistent Python interface** (Model defines equations and events, Solver defines how the problem is solved)
- Assimulo support **discontinuous systems** via:
 - State events: zero-crossing functions -> contact
 - Time events: discrete changes at specific times
 - Step events: reparameterization without simulation
- **Event handling** requires
 - event indicator functions
 - restarting solver on events
 - Optional simplifications when localization is not required
- Models are implemented as **Cython/Python classes**



PyFMI: FMI Wrapper and Co-Simulation Engine

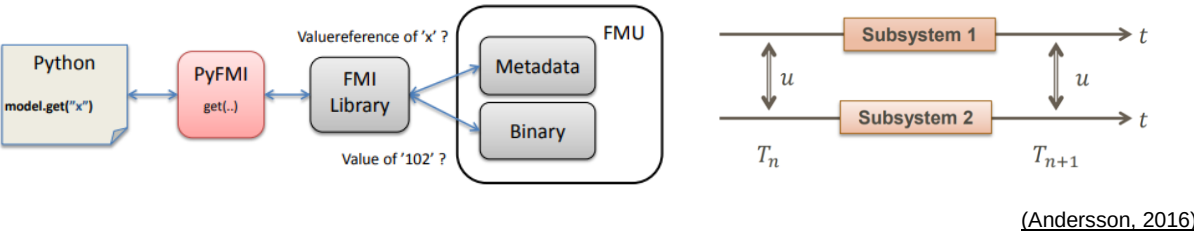
- Enables simulation of FMUs(FMI 1.0 & 2.0) within Python environments
- It acts as a runtime engine that can load FMUs and interact with their internal variables using Python, enabling scripting, automation, post-processing, and integration into Python-based workflows

Key Concepts

Concept	Description
FMUs	• Supports FMI for Model Exchange and Co-Simulation • Can load and handle .fmu files
Simulation	• Set simulation options (step size, solver type) • Run simulations (Model Exchange with solvers provided by Assimulo) • Handle simulation results
Variable Access	• Get/Set values of parameters, inputs, outputs and internal variables • Access derivatives, state variables, and event indicators (for ME FMUs) • Use variables via name and value reference
Events & Discontinuities	• Handles state events and time events in Model Exchange FMUs • Use event iteration to locate and handle discontinuities

Challenge: Hybrid Co-Simulation using PyFMI

- PyFMI does not support FMI 3.0 and its additional features for hybrid Co-Simulations
 - **Intermediate update:** FMU can interact during a step with the importer (event handling)
 - **Early Return:** FMU signal internal event, importer can act before completing step
- Without feature, PyFMI cannot detect or react to internal events or signal to the FMU that an external event causes a restart of the integration step
- Precise event detection only with Model Exchange (or alternative FMPy by Dassault Systèmes)



(Andersson, 2016)

Core Modeling Concepts

- Continuous and discrete systems modeled via ODEs, difference equations, transfer functions, state-space models
- Block-based modeling (causal, user-defined): visual hierarchies and data flow structures
- Models as sequence of function blocks: $y = f(u)$
- Look-up tables: interpolated, multidimensional modeling of functions
- Virtual and atomic subsystems help with modularization and execution control
- Hybrid systems: supported with Stateflow or Simscape events

Simulation Management

- Solvers handle the numerical integration (fixed and variable step)
- Performance tuning via solver profiler and zero-crossing detection
- Conditional execution for event-driven modelling: enabled and triggered

Data Handling & Integration

- Exporting and importing of data: .mat, workspace, spreadsheets
- Signal organization via Bus signals
- Scope and data inspector for output analysis
- Model referencing supports scalability and modularity

Testing & Modularity

- Model referencing enables reusable component design with strong encapsulation
- Support for unit testing, integration testing, and simulation comparison
- Workspaces isolate and manage parameters for modularity and reuse

User Interface & Visualization

- Masking abstracts the complexity and exposes parameters in a dialog
- Live analysis via scope, port labels, data inspectors
- Dependency analyzer maps model references and hierarchies

Relevant Concepts

Concept	Description
Visual hierarchy of subsystems	• Sub model encapsulation • Modular simulation blocks (Python classes for subsystems like Motor Subsystem) • Subsystem templates with input configuration-file
Atomic subsystems	• Execution controlled modules • For FMI components
Bus Signals	• Typed and grouped signal interface
Workspaces	• Base, model, mask workspaces • Enable scoped configuration for project, model, and component level
Conditional Subsystems	• Event-driven execution • Trigger-based simulation paths
Model referencing	• Modular and reusable sub models
Masking	• Parameter dialog • Differentiation of standard- vs. expert-user interface
Scopes and Data Inspectors	• Live plotting and result comparison tool • Integration of real-time insights (matplotlib, Plotly)
Architecture	• Layered architecture • Clean separation between model description, simulation engine, and user interaction
Tools	• Debugging complex models with dependency analysis tool • Simulation profiling: identifying performance bottlenecks • SSA: approximation models

(MathWorks, 2025)

Overview

- System Designer Requirements: Requirements that benefit the system designers using the framework
 - Making environment easy to use
 - Managing large amount of system information
 - Visualize complex system behavior
- Architectural Requirements: requirements related to the creation and maintenance of the simulation environment / framework (keeping software complexity manageable)
 - Clarity of structure, maintainability, flexibility, code reuse
 - Affect system designers only indirectly but with significant impact on the software architecture of the framework
- Execution modes of the framework:
 - Manual control or invoked automatically
 - Treated as black box (co-simulation) or as a network of more detailed lower-level simulators
 - User can use full simulation model, or a much faster approximation generated by the framework

Managing Software Complexity

- Decomposition of the simulator into submodules that interact by exchanging information
- Framework manages the data flow
- Generic services are extracted from the simulation modules into the framework layer

Easy-to-use Simulation Environment

- System designers must be able to set any design parameters that can be adjusted
- System may confront the users with vast quantities of intermediate and output data
- Visualization of sub models and their network and data dependencies
- High-level user interface based on data flow paradigm
 - tool status represented via colors
 - set of sub models can be combined to form a higher-level sub model
 - intermediate simulation results can be examined

Subspace Approximation (SSA) Models

- Use an approximation instead of the full model with a reduced set of relevant factors
- Faster design cycle may outweigh loss of accuracy
- Software service to automate the generation of SSA models (approximation is automatically derived from the full system model on demand)
- SSA is generated and valid for a restricted parameter subspace

Framework: Architectural Approach

- Decomposition of big simulation tools into relatively small modules representing well-defined submodules
- Sub models should be implemented or adapted by the sub model experts
- Modularization along expertise boundaries: close gap between developers (know-how about product technologies) and IT professionals (expertise in developing complex software)
- Availability of a graphical representation of the data flow stimulates the architectural awareness
- File based data flow mechanism to integrate the sub models
- Rigorous form of modularization enhances clarity and flexibility
- Framework can provide various extra services because it knows how the tools work together

Characteristics and Benefits

- System model is partitioned into a network of tools that exchange files/data
- Framework invokes and monitors tool runs and manage all system design files
- Additional services support visualizing and analyzing the behavior of a parameterized system
- Entire framework is generic and application independent
- Services attempt to solve a set of generic problems which are relevant in a wide range of system design environments

(van den Hamer et al., 1999)

V-Model for System Development

- Verification and Validation Model widely used for software and systems
- Emphasizing testing and validation at each development stage

Key concepts:

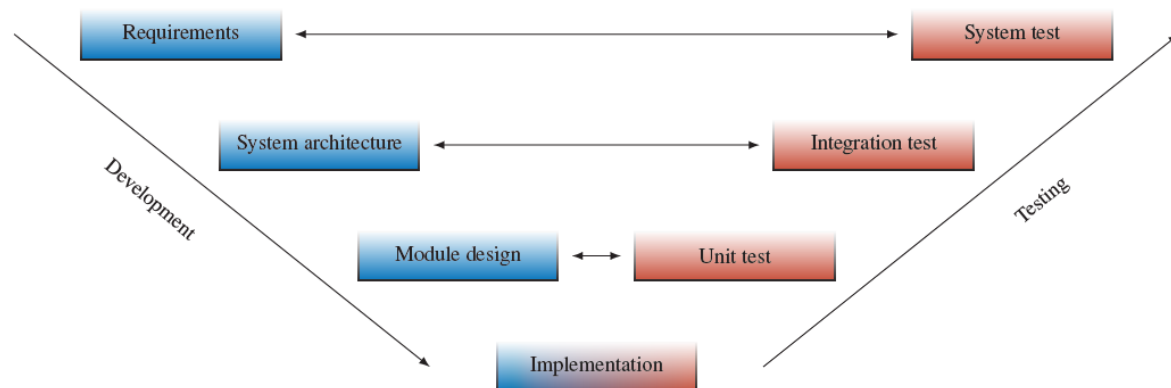
- highlights a correspondence between development phases and their associated testing phases
- Ensuring that for every stage of development, there is a matching validation phase

Left Side: Development

- Requirement Analysis: Gather what the system should do
- System Design: High-level design and system architecture
- Architectural Design: Break the system into modules/components
- Module Design - Specify how each module will function
- Coding/Implementation - Write the actual code

Right Side: Testing

- Unit Testing: Verify that each module works correctly
- Integration testing: Check interactions between modules
- System testing: Validate the complete system against the requirements
- Acceptance testing: Confirm that the system meets user needs



Requirements and Testing

- **Requirement** is a need or expectation that is stated, generally implied or obligatory
- **Component** is any subset in the system architecture of the simulation framework
- **Quality** is the capability of a component to satisfy stated and implied needs when used under specific condition
- **Quality evaluation** is a systematic examination of the extent to which a component can satisfy stated needs and implied needs
- **Validation:** Tests confirming and providing objective evidence that the requirements for a specific intended use or application have been fulfilled
- **Verification:** Tests confirming and including objective evidence that specified requirements have been fulfilled.

Derivation of Test Cases

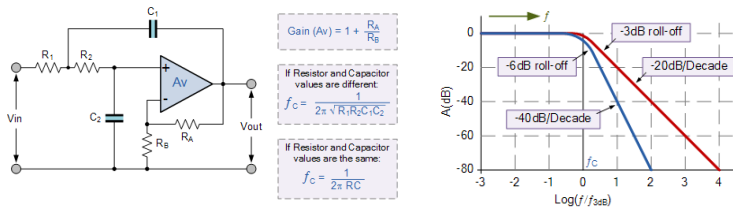
1. **Identify unit:** function, methods, classes or components
2. **Understand requirements:** clear needs help define scope and purpose of unit test
3. **Define test:** one test with one test set and one quality per requirement
4. **Determine input/output:** limit range of inputs and outputs
5. **Write test code:** utilizing testing frameworks
6. **Execute tests:** run to validate correctness
7. **Analyze result:** check validity and identify errors/failures to investigate causes
8. **Refine:** make steps towards verification
9. **Maintain coverage:** unit test reflect standard behavior checks as the code gets larger

Principles of Continuous Integration (CI)

1. **Maintaining a single source repository**
 - single, central repository
2. **Automating the build process**
 - ensure code is consistently automatically built
3. **Build in a clean environment**
 - eliminate dependencies
4. **Keep builds fast**
 - Obtain quick feedback
5. **Run automated tests**
 - unit, integration, regression tests
 - verify correctness and quality before propagate further
6. **Fail fast**
7. **Provide feedback**
8. **Practice continuous integration**

(Pannek, 2023)

Low-Level: 2nd Order Low Pass Filter



Description:

- Equation-based model: can be described by ODEs or DAE with nodal potentials
- Modularity: clearly separated subcomponents for resistance, capacitor and operational amplifier
- Variability of parameters
- Continuous-time behavior

Components:

- RC-elements
- Operational amplifier
- Feedback loop
- Input and output interface

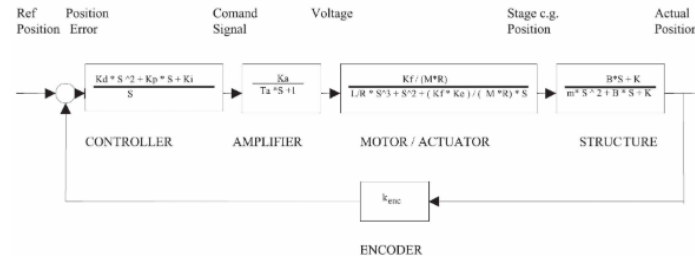
Features for Simulation Framework:

- Equation-based modeling
- Feedback within the system
- Replacement of subsystems
- Parameter variation via config files

Purpose: Comparison with existing framework

(Filter zweiter Ordnung | Tiefpassfilter zweiter Ordnung)

Medium-Level: Mass Positioning



Description:

- Mass is positioned by an actuator
 - Electrical drive (equation-based)
 - Muscle actuator (musculoskeletal subsystem)
- Elastic element couples actuator and mass
 - simplified elastic element (equation-base)
 - FEM-based subsystem

Subsystems:

- Mechanical Subsystem
- Actuator Subsystem
- Elastic Coupling Subsystem
- Sensor and Control Subsystem

Features for Simulation Framework:

- Replacement of equation-based subsystems by FEM-based or musculoskeletal subsystems
- Event-triggered dynamics via sensor and controller

Purpose: Simple system with heterogenous sub models

(Motion Control Systems - Parker Hannifin)

High-Level: Leg Orthosis C-Brace (simplified)



Description:

- Interaction with musculoskeletal system (different stages of partial paralysis)
- Sensors for leg motion detection (angular position and direction of motion)
- Controller uses sensory data to predict required support through actuator (actuator activation) in real time
- Hydraulic actuator acts on mechanical structure

Subsystems (FMUs):

- Human leg with muscles, segments, joints (gait trajectory)
- Hydraulic actuator
- Controller

Features for Simulation Framework:

- Complex hybrid simulation with event handling
- Replacement of sub models (OpenSim) for musculoskeletal leg model and/or actuator as FEM model (structural analysis of mechanical components)

Purpose: Complex hybrid system with multiple heterogenous sub models

(C-Brace - Die computergesteuerte Beinorthese)

1. Software Engineering und Framework Design

- Requirements Engineering
- System Modeling
- Architectural Design
- Design and Implementation
- Software Testing
- SOLID Design Principles
- Design Patterns

2. Anforderung & Zieldefinition

- Ableitung funktionaler und nicht-funktionaler Anforderungen (orientiert am V-Modell)
- Analyse und Bewertung des bestehenden Frameworks hinsichtlich Modularität, Erweiterbarkeit und Toolkopplung
- Abgrenzung des Umfangs und Festlegung des angestrebten Entwicklungsziels

3. Demo-Modelle

- **Demo Systeme konkretisieren**
 - **Welche Framework-Funktionalitäten müssen abgedeckt werden?**
- Framework / Architektur Draft für Implementierung des Low-Level Demo Modells
- Implementierung des ersten Demo-Modells
 - Pure Python ohne externe Tools
 - Simulink
 - Modelica (Export Subsystem als FMUs für Co-Simulation / Model Exchange)
 - OMPython
 - FMPy

4. Architekturplanung und Konzeptentwicklung

- Welche Tools können für welche Funktionen eingesetzt werden?
- Welche externen Schnittstellen sind erforderlich?
- Iterativer Aufbau anhand Demo-Modellen
- Entwicklung Subsystem-Schnittstellen & Software Modulgrenzen

1. Andersson, C. (2016). **Methods and tools for co-simulation of dynamic systems with the Functional Mock-up Interface** [Doctoral thesis, Lund University].
2. Cremona, F., Lohstroh, M., Broman, D., Lee, E., Masin, M., & Tripakis, S. (2018, October). **Hybrid co-simulation: It's about time**. In Proceedings of the 21st ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (pp. 368). <https://doi.org/10.1145/3239372.3242896>
3. Fritzson, P. (2015). **Principles of object-oriented modeling and simulation with Modelica 3.3: A cyber-physical approach** (2nd ed.). Wiley-IEEE Press.
4. MathWorks. (2025). **Simulink documentation**. <https://www.mathworks.com/help/simulink/>
5. Modelica Association Project "FMI". (2022, October). **Functional Mock-up Interface Specification**, Version 3.0. <https://fmi-standard.org/>
6. Pannek, J. (2023, July 18). **Simulation of mechatronic systems (Simulation mechatronischer Systeme)** [Lecture notes]. Technische Universität Braunschweig.
7. Sima, H. (2024). 317.047 **Computational Bionics: Modelling and Simulation** [Lecture slides]. Institute of Lightweight Design and Structural Biomechanics, TU Wien. <https://www.ilsb.tuwien.ac.at>
8. Urquía Moraleda, A., & Martín Villalba, C. (2018, April). **Modeling and simulation in engineering using Modelica**. Editorial UNED. <https://e-spacio.uned.es/entities/publication/cbfb24a9-2ec0-4afa-aa9c-243e1309da6e/full>
9. van den Hamer, P., van der Linden, W. P. M., Bingley, P., & Schellingerhout, N. W. (1999). **A system simulation framework: Software environments for designing complex products**. Philips Research Laboratories Eindhoven.
10. Wetter, M. (2019, February 15). **Functional Mock-up Interface** [Presentation]. Simulation Research Group, Lawrence Berkeley National Laboratory.